Images, media, and embedded content

The web is more than text. Pages show photos, diagrams, videos, audio, and content embedded from other sites. HTML has dedicated elements for these, and most of them follow the same basic pattern: tell the browser where the media lives, then provide useful text for people or browsers that cannot use the media directly.

That useful text looks different depending on the element. For images, it is usually `alt` text. For video and audio, it can be fallback text between the opening and closing tags. For iframes, it is a `title` that labels the embedded page.

Images: the `<img>` element

The `<img>` element loads an image into the page. It is a void element, which means it has no content and no closing tag. It relies on two essential attributes.

```html
<img src="cat.jpg" alt="A black cat sitting on a windowsill">
```

The `src` attribute is the path to the image. It can be an absolute URL, a root-relative path, or a relative path.

The `alt` attribute is text that describes the image. Leaving out `src` means the image will not load. Leaving out `alt` means users who cannot see the image may lose the information it carries.

Alt text: write it for someone who cannot see the image

Alt text is what screen readers read aloud, what search engines can use, and what the browser shows when the image fails to load. Good alt text describes the image's purpose in the context of the surrounding content.

For an informative photo:

```html
<img src="bike-route.jpg" alt="A flat dirt path through pine trees">
```

For an image that conveys data, like a chart or diagram:

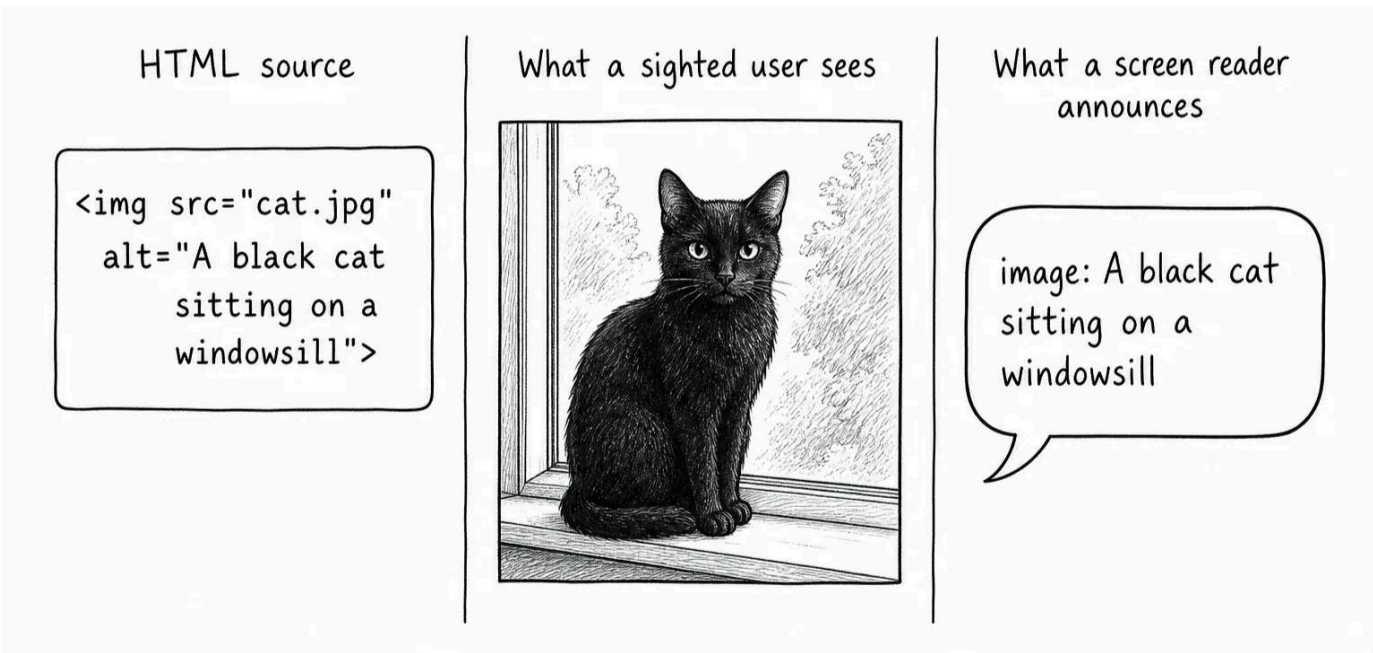
```html
<img src="sales-chart.png" alt="Bar chart showing 2025 sales doubling from Q1 to Q4">
```

For a purely decorative image, one that adds visual flair but no information, use an empty `alt` attribute. Screen readers will skip it.

```html
<img src="decorative-swirl.svg" alt="">
```

Empty `alt` is intentional. Leaving the attribute off entirely is different: assistive tools then have nothing to work with and may read the filename instead. Always include the `alt` attribute.

> **Write alt like a caption** - If the image were replaced by your alt text, would the page still make sense? If yes, your alt text is doing its job.

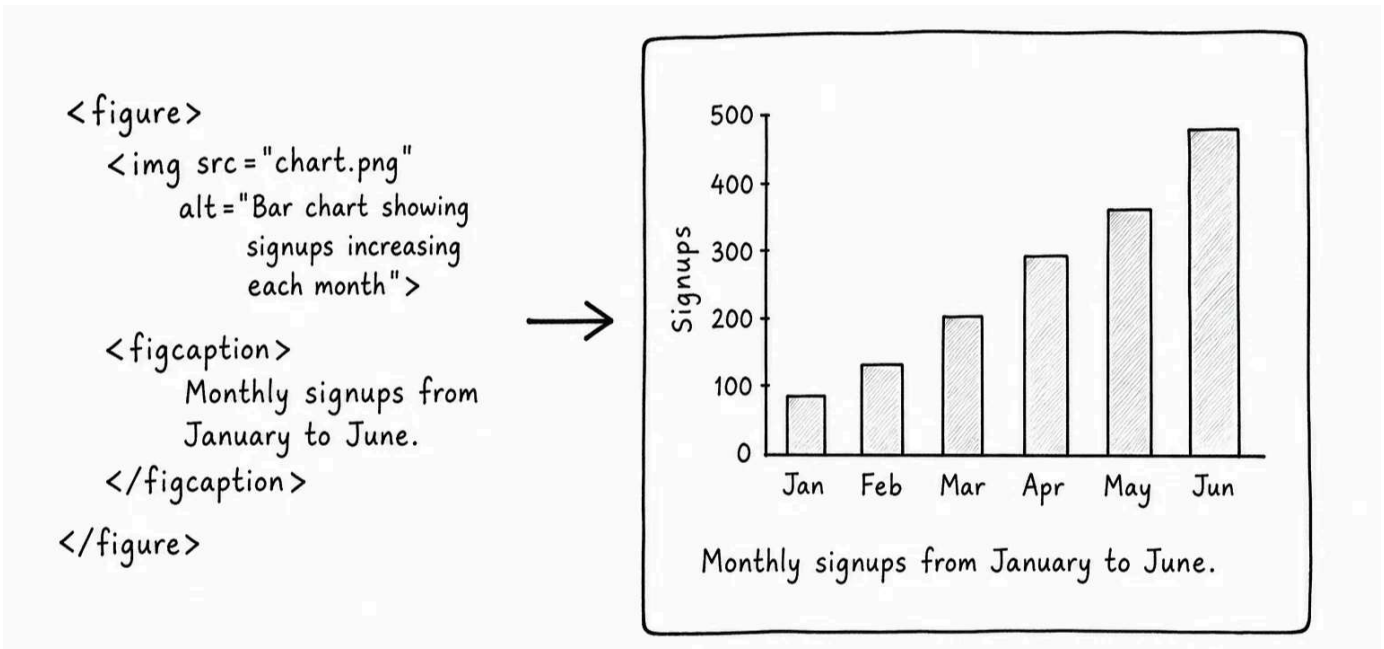


Figures and captions

When an image needs a visible caption, wrap it in `<figure>` and put the caption in `<figcaption>`.

```html
<figure>
  <img src="chart.png" alt="Bar chart showing signups increasing each month">
  <figcaption>Monthly signups from January to June.</figcaption>
</figure>
```

The `alt` text and the caption are not the same job. The `alt` text stands in for the image when someone cannot see it or when the image fails to load. The caption is visible page content that gives extra context to everyone.



Dimensions and layout shift

Set the width and height of an image when you know them:

```html
<img src="hero.jpg" alt="..." width="1200" height="600">
```

These attributes do two things. They reserve space in the layout while the image is downloading, which prevents the page from jumping around when the image arrives. They also give the browser the image's aspect ratio so CSS can scale it without distortion.

You can still resize the displayed image with CSS. The attributes are about reserving space, not locking the pixel size.

If an image is far down the page, you can also add `loading="lazy"`:

```html
<img src="gallery-1.jpg" alt="A product photo" width="600" height="400" loading="lazy">
```

This tells the browser it can wait to load the image until the user gets closer to it. Do not use lazy loading for the main image at the top of the page, because that image should load right away.

Image formats

The format you choose affects file size and qkuality.

Format	Best for
`JPG` / `JPEG`	Photographs and rich images with smooth gradients
`PNG`	Logos, screenshots, or images that need transparency
`WebP`	A modern format that often beats JPG and PNG on file size

Format	Best for
`SVG`	Icons, logos, and diagrams that should scale crisply at any size
`GIF`	Short looping animations (often better as a video today)

Browsers load all of these the same way. A safe beginner default: JPG for photos, PNG for screenshots, SVG for icons.

Responsive images, briefly

Sometimes you need to serve different image sizes to different screens. A phone does not need the same huge hero image as a large desktop monitor. HTML has `srcset` and `<picture>` for this:

```html
<img src="hero.jpg" srcset="hero-2x.jpg 2x" alt="...">
```

You do not need the details yet. Just know these exist for responsive images. They become useful when image performance starts to matter.

AI Tutor Explain responsive images and srcset with a small beginner example.

Video and audio

The `<video>` and `<audio>` elements work similarly to `<img>`, but unlike `<img>`, they have content and a closing tag.

```html
<video src="intro.mp4" controls width="640" height="360" poster="intro-cover.jpg">
  Your browser does not support video playback.
</video>
```

Two attributes matter right away.

- `controls` tells the browser to show play, pause, and volume controls. Without it, the video has no visible controls.
- `poster` is a placeholder image shown before the video starts playing.

The text between the tags is fallback content, shown only when the browser cannot play the video at all.

For audio, the pattern is similar but simpler:

```html
```

```
<audio src="podcast.mp3" controls>
  Your browser does not support audio playback.
</audio>
```

If you need to provide more than one file format, use `<source>` elements inside the video or audio:

html

```
<video controls>
  <source src="intro.webm" type="video/webm">
  <source src="intro.mp4" type="video/mp4">
</video>
```

The browser picks the first source it can play.

Embedded content with `<iframe>`

An `<iframe>` lets you embed another HTML document inside the current one. Common examples include a YouTube video, a map, or a payment widget from a third-party service.

html

```
<iframe src="https://www.youtube.com/embed/abcd1234"
        width="560"
        height="315"
        title="Intro video">
</iframe>
```

For iframes, `title` plays the accessibility role that `alt` plays for images. It gives screen readers a useful label for the embedded content.

⚠ **Be careful what you embed** - An iframe loads another page inside yours. Only embed content from sources you trust. Browsers add some isolation around iframes, but they cannot make an untrusted embedded page safe.

You probably will not write many iframes by hand. When you do, it is usually a snippet given to you by a service like YouTube, Vimeo, or Stripe. Paste it, give it a meaningful `title`, and move on.

When media comes from users or app data

Real apps often deal with media that users upload or that the server serves back later. A few habits matter here:

- **Validate uploaded images on the server**, not just in the browser. Check file type, size, and dimensions before storing. The `accept` attribute on a file input is a UX hint, not a security boundary.

- **Strip image metadata** when users upload photos. EXIF data can include GPS coordinates and device details that the uploader may not want shared.
- **Set cache headers** on media. Images, video, and audio can be expensive to download. A long `Cache-Control: max-age` on fingerprinted filenames is the usual approach.

AI Tutor What is EXIF data in images? >

Try it

Save any image as `photo.jpg` in a folder. Then save this file as `media.html` in the same folder and open it in your browser:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Media practice</title>
  </head>
  <body>
    <h1>Media practice</h1>

    <img src="photo.jpg" alt="Describe what is actually in your photo" width="320">

    <video src="sample.mp4" controls width="480" poster="photo.jpg">
      Your browser does not support video playback.
    </video>
  </body>
</html>
```

Try a few changes and reload between each:

- Change `alt` to an empty string `""`. Nothing visible changes for you, but a screen reader will now skip the image.
- Remove `width="320"` from the image. The image now shows at its natural size.
- Remove `controls` from the video. The player UI disappears even though the video element is still there.
- Change `poster` to point at a different image, or delete the attribute. Notice what the player looks like before it would start playing.
- Change the fallback text inside `<video>`. You probably will not see it in a modern browser, but it is still part of the HTML for browsers that cannot play the video.

You do not need a real `sample.mp4` file for most of these changes to be visible. The player UI is created from the HTML attributes. Open DevTools and

inspect the `<img>` and `<video>` nodes. Both are DOM elements with attributes, just like the elements you have already seen.